Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251191**
Nama Lengkap	**VIRJIE GRACIA TUMIMOMOR**
Minggu ke / Materi	**09/Pengolahan String dan Regular Expression**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

MATERI 1

Pengantar String

Stirng adalah untaian karakter" yang menjadi satu kesatuan dan digunakan dalam program komputer untuk menyimpan kalimta Panjang atau pendek, dan string adalah suatu jeni data yang mampu menyimpan huruf/karakter dan disimpan dalam kode ASCII. Tapi dalam bahasa C tidak memakai tipe data string.

Pengaksesan String dan Manipulasi String

Stirng dapat dibuat dengan sederhana menggunakan variable, berikut pada Gambar 9.1:

```
namasaya = "Virjie Gracia Tumimomor"
temansaya1 = "Cheila Zefanya Wibowo"
temansaya2 = 'Alicia Luna Santoso'
temansaya3 = "Angela Revaline Setiawan"
print(temansaya3)
print(namasaya[0])
print(namasaya[9])
print(temansaya1[1])
huruf = temansaya2[0]
print(huruf)
```

Gambar 9.1: Tampilan menggunakan variable.

Jadi string pertama kali dibuat dengan deklarasi variable dan langsung diisi dengan data string dapat diakses sebagai satu kesatuan dengan menyebut nama variblenya, atau per huruf dengan menyebutkaan indeksnya. Dan indeks pada string di mulai dari 0 seperti layaknya pada list dan indeks pada string harusalah bilangan bulat, bukan pecahan. Berikut contoh pada Gambar 9.2:

b	a	n	a	n	a
[0]	[1]	[2]	[3]	[4]	[5]

Gambar 9.2: Tampilan Bentuk String di dalam memory.

Operator dan Metode String

OPERATOR in

Pada String kita dapat memeriksa apakah suatu kalimat merupakan substring dari suatu kalimat lain dengan menggunakan operator in. Hasil dari operator ini adalah True / False. Berikut pada Gambar 9.3:

```
kalimat = "saya mau makan"
data = "saya"
print(data in kalimat)
print("mau" in kalimat)
print("dia" in kalimat)
```

Gambar 9.3: Tampilan operator True/False.

```
if "saya" > "dia":
    print("Ya") #Ya
else:
    print("Tidak")
if "dua" == "dua":
    print("Sama")
```

Gambar 9.4: Perbandingan(comporsion) yang juga menghasilkan True/False.

FUNGSI len

Cara untuk mengetahui berapa Panjang atau jumlah karakter dari sebua string adalah dengan menggunakan operator len (<string>) untuk menampilkan huruf terakhir dari sebuah string menggunakan ideks string yang ke- len (<string>-1), karena indeks di mulai dari 0. Berikut contoh pada Gambar 9.5:

```
kalimat = "universitas kristen duta wacana yogyakarta"
print(len(kalimat))
terakhir = kalimat[len(kalimat)-1]
print(terakhir)
#bisa juga menggunakan indeks-1
terakhir_versi2 = kalimat[-1]
print(terakhir_versi2)
#atau menggunakan indeks-2 untuk huruf terakhir kedua
terakhir2 = kalimat[-2]
print(terakhir2)
```

Gambar 9.5: Tampilan dalam python.

TRAVERSING STRING

Kita menggunkan loop untuk menampilkan huruf demi huruf.

- Dilakukan dengan akses terhadap indeks

```
kalimat = "indonesia jaya"
i = 0
while i < len(kalimat):
    print(kalimat[i],end='')
    i += 1
```

- Dilakukan tanpa akses terhadap secara otomatis

```
kalimat = "indonesia jaya"
for kal in kalimat:
    print(kal,end='')
```

STRING SLICE

Untuk menampilkan substring pada sebuah stirng dengan menggunakan indeks dari awal tertentu sampai akhir -1 tertentu. Bagian awal atau akhir bisa di kosongkan, dan awalnya di mulai dari 0.

```
kalimat = "cerita rakyat"
awal = 0
akhir = 6
print(kalimat[awal:akhir])  #cerita
print(kalimat[7:len(kalimat)])  #rakyat
print(kalimat[:5])  #cerit
print(kalimat[5:])  #a rakyat
print(kalimat[:])   #cerita rakyat
```

Berikut adalah beberapa method string yang sering digunakan. Pada Gambar 9.6:

Nama Method	Kegunaan	Penggunaan
capitalize()	untuk mengubah string menjadi huruf besar	string.capitalize()
count()	menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	mengetahui apakah suatu string diakhiri dengan string yang dinputkan	string.endswith()
startswith()	mengetahui apakah suatu string diawali dengan string yang dinputkan	string.startswith()
find()	mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() dan isupper()	mengembalikan True jika string adalah huruf kecil / huruf besar	string.islower() dan string.isupper()
isdigit()	mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	menghapus semua whitespace yang ada di depan dan di akhir string	string.strip()
split()	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	string.split()

Gambar 9.6: Tampilan method yang sering digunakan.

Dan daftar lengkapnya bisa di lihat di https://docs.python.org/library/stdtypes.html#string-methods.

Operator * dan + pada String

Operator * dan + memiliki kemampuan khusus, + di gunkan untuk menjumlahkan bilangan bisa digunkan untuk menggabungkan dua buang string. Dan * yang biasa di gunakan untuk mengkalikan bilangan bisa digunakan unutk menampilkan string sejumlah perkaliannya. Perhatikan kode berikut pada Gambar 9.7:

```
kata1 = "aku"
kata2 = "lapar"
kata3 = kata1 + " " + kata2
print(kata3)
kata4 = "ambon manise "
print(kata4 * 4)
kata4 = "manis "
print(kata4 * 2)
```

Gambar 9.7: Tampilan kode pemrograman.

Parsing String

Parsing string adalah cara menelusuri string bagian untuk mendapatkan/menemukan/ mengubah bagian string yang diinginkan. Perhatikam contoh berikut:

“Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka". Dilihat dari string di atas, diingikan untuk mengambil tanggal bulan tahun dari kalimat di atas dan disusun ulang menjadi format 08/17/1945. Kita dapat menggunakan cara parsing string misalnya:

1. Spit string berdasarkan spasi sehingga menjadi token: "Saudara-saudara", "pada", "tanggal", "17-08-1945", "Indonesia", dan "merdeka".
2. Dari sini, lanjutkan dengan mencari token yang diawali dengan angka, kemudian lanjutkan dengan split angka tersebut dengan pemisah ’-’.
3. Kemudian susun ulang token-token dari langkah sebelumnya sehingga berbentuk seperti yang diinginkan. Berikut adalah contoh penyelesaiannya:

```
kalimat = "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka"
hasil = kalimat.split(" ")
for kal in hasil:
    if kal[0].isdigit():
        hasil2 = kal.split("-")
        print(hasil2[1]+"/"+hasil2[0]+"/"+hasil2[2])
```

Gambar 9.8: Tampilan penyelesaian.

Dan output dari itu semua **08/17/1945**.

Pengantar Regex

Pada bab mengenai string kita sudah banyak belajar mengenai strimh, bahwa kita cukup kesulitan unutk melakukan pengolahan string dengan Teknik biasa/ standar. Tapi ada juga cara yang lebih mudah dan cepat dengan menggunkan bantuan *Regular expression,* adalah ekspresi pola yang berbentuk kumpulan karakter yang digunakkan unutk menemukan pola(pattern) yang sama dengan pola regex di dalam string lain yang ingin daicari. Regex digunakan untuk mencari, mengganti, dan menghapus teks berdasarkan pola tertentu dalam string. Dibandingkan fungsi biasa seperti split() dan find(), regex lebih powerful meskipun polanya cukup rumit. Python mendukung regex melalui library re, salah satu fungsi yang mudah digunakan adalah search().

Contohnya, kita bisa menggunakan file *mbox-short.txt* untuk menampilkan semua baris yang **mengandung** pola **"From: "**

```
import re
handle=open('mbox-short.txt')
count = 0
for line in handle:
line=line.rstrip()
if re.search('From:', line):
count += 1
print(line)
print("Count: ",count)
```

Gambar 9.9: Tampilan string mengandung **From**

Dari kode diatas bahwa re.search bisa di Ganti dengan menggunkan peritah **find**() pada string.

```
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
From: gsilver@umich.edu
From: gsilver@umich.edu
From: zqian@umich.edu
From: gsilver@umich.edu
From: wagnermr@iupui.edu
From: zqian@umich.edu
From: antranig@caret.cam.ac.uk
From: gopal.ramasammycook@gmail.com
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: louis@media.berkeley.edu
```

Gambar 9.10: Tampilan beberapa from.

Jika diinginkan mencari baris yang diawali denga pola “From”, maka kita harus mengubah parameter fungsi search pada re.search menajdi **re.search(“From”).**

```
import re
handle=open('mbox-short.txt')
count = 0
for line in handle:
line=line.rstrip()
if re.search('^From:', line):
count += 1
print(line)
print("Count: ",count)
```

Kode di atas dapat di gantikan juga dengan fungsi string.startswith(“From:”).

Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Libary Python

Sebelum menggunakan fungsi regex perlu diketahui terlebih dahuli meta character/special character dan kegunaannya pada pola regex seperti pada Gambar berikut:

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

Gambar 9.11: Tabel Himpunana Karakter pada Regex.

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{}	Karakter dengan arti khusus dan escaped character	\{}d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
$	Dakhiri dengan	this$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{}d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{}d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{}d+	Minimal 1 s/d tak terhingga karakter
{}	Tepat sebanyak yang ada para {}	\{}d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(sayalkamu)	saya atau kamu sebagai satu kesatuan
|	atau	\{}d|\{}s	1 digit atau 1 spasi

Gambar 9.12: Tabel* Special Character pada Python.

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\bin" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

Gambar 9.13: Tabel Escaped Character pada Regex.

Nama Fungsi	Kegunaan
findall	mengembalikan semua string yang sesuai pola (matches)
search	mengembalikan string yang sesuai pola (match)
split	memecah string sesuai pola
sub	mengganti string sesuai dengan pola yang cocok

Gambar 9.14: Tabel fungsi Regex

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link GitHub : https://github.com/virjiegracia/71251191VIRJIE-GRACIA.git

SOAL 1

SOURCE CODE:

```
def cek_anagram(kata1, kata2):
    kata1 = sorted(kata1.lower())
    kata2 = sorted(kata2.lower())

    if kata1 == kata2:
        return "Anagram"
    else:
        return "Tidak Anagram"

kata1 = input("Masukkan kata pertama: ")
kata2 = input("Masukkan kata kedua: ")

print("Hasil:", cek_anagram(kata1, kata2))
```

INPUT:

```python
def cek_anagram(kata1, kata2):
    kata1 = sorted(kata1.lower())
    kata2 = sorted(kata2.lower())

    if kata1 == kata2:
        return "Anagram"
    else:
        return "Tidak Anagram"

kata1 = input("Masukkan kata pertama: ")
kata2 = input("Masukkan kata kedua: ")

print("Hasil:", cek_anagram(kata1, kata2))
```

OUTPUT:

```
Masukkan kata pertama: atma
Masukkan kata ked
Hasil: Anagram
14-64\python.exe c:/Users/Lenovo/al
Masukkan kata pertama: lala
Masukkan kata kedua: alal
Hasil: Anagram
PS C:\Users\Lenovo\alpro>
```

LANGKAH PENYELESAIA:

1. Pertama membuat fungsi dengan cek_anagram dengan parameter kata1 dan kata2.
2. Kata1 dan kata2 menggunakan fungsi lowe untuk mengubah huruf menajadi kecil.
3. Dan sorted untuk mengembalikan daftar huruf yang sudah di urutkan dari kecil ke besar.
4. Lalu membandingankan kedua daftar huruf tersebut. Di cek apakah anagram jika benar maka keluar Anagram dan selain itu berarti Tidak anagram/
5. Dan untuk input kata1 dan kata2 untuk di lakukan perbandingkan.
6. Lalu print untuk menampilkan dengan cek apa angram atau bukan.

SOAL 2

SOURCE CODE:

```python
kalimat = input("Masukkan kalimat: ")

kata_cari = input("Masukkan kata yang dicari: ")


kalimat = kalimat.lower()

kata_cari = kata_cari.lower()


daftar_kata = kalimat.split()
```

```
jumlah = daftar_kata.count(kata_cari)

print(f"{kata_cari} ada {jumlah} buah")
```

INPUT:

```
kalimat = input("Masukkan kalimat: ")
kata_cari = input("Masukkan kata yang dicari: ")

kalimat = kalimat.lower()
kata_cari = kata_cari.lower()

daftar_kata = kalimat.split()

jumlah = daftar_kata.count(kata_cari)

print(f"{kata_cari} ada {jumlah} buah")
```

OUTPUT:

```
Masukkan kalimat: saya lapar . lapar sekali. karena lapar makanya aku ke cafet
Masukkan kata yang dicari: lapar
lapar ada 3 buah
PS C:\Users\Lenovo\alpro>
```

LANGKAH PENYELESAIAN:

1. Pertama buat dua variable untuk menginput data yang di inginkan. Kalimat dan kata yang akan di cari dalam kalimat.
2. Lalu pada kalimat itu mnggunkan fungsi lower yang di mana nantinya jika kita menginput huruf besar dia tetap terdeteksi.
3. Lalu variable daftar kata untuk memechkan suatu string dengan koma agar bisa di bedakan.
4. Lalu pada jumlah menggunakan fungsi count unutk mengecek dalam daftar kata berapa kali suatu kata muncul dalam sebuah objek.
5. Lalu print untuk menampilkan hasil.

SOAL 3

SOURCE CODE:

```
def hsb(kalimat):
    return " ".join(kalimat.split())
```

```
kata= input("masukan kalimat:")
hasil=hsb(kata)

print(hasil)
```

INPUT:

```
def hsb(kalimat):
    return " ".join(kalimat.split())

kata= input("masukan kalimat:")
hasil=hsb(kata)

print(hasil)
```

OUTPUT:

```
masukan kalimat:saya  lapar sekali  karena lagi  puasa
saya lapar sekali karena lagi puasa
PS C:\Users\Lenovo\alpro>
```

LANGKAH PENYELESAIAN:

1. Pertama saya membuat fungsi hapus spasi berlebihan yang saya singkat jadi **hsb** dan 1 parameter yaitu kalimat.
2. Lalu return di sini ada split untuk memecahkan suatu string.
3. Lalu **" ".join** untuk mengabungkan Kembali list kata dengan menggunkan satu spasi.
4. Variable **kata** untuk input kalimat yang inngin di cek.
5. Lalu print hasil.

SOAL 4

SOURCE CODE:

```
kalimat = input("Masukkan kalimat: ")

kata_list = kalimat.split()

terpendek = min(kata_list, key=len)
```

```
terpanjang = max(kata_list, key=len)


print("Terpendek:", terpendek)
print("Terpanjang:", terpanjang)
```

INPUT:

```
kalimat = input("Masukkan kalimat: ")

kata_list = kalimat.split()

terpendek = min(kata_list, key=len)
terpanjang = max(kata_list, key=len)

print("Terpendek:", terpendek)
print("Terpanjang:", terpanjang)
```

OUTPUT:

```
Masukkan kalimat: sa ini lapar sekali
Terpendek: sa
Terpanjang: sekali
PS C:\Users\Lenovo\alpro>
```

LANGKAH PENYELESAIAN:

1. Variable kalimat unutk menginput kalimat yang di masukan oleh pengguna.
2. Lalu pada bagian kata_list untuk memcahkan kalimat berdasarkan spasi.
3. Variable terpdendek untuk mengecek kata paling pendek begitu sebaliknya dengan terpanjang.
4. Dan **key** untuk menentukan ukuran atau Panjang untuk setiap kalimat =len yang artinya apakah key itu sama Panjangnya dengan len? Jika tidak makan bisa jadi yang terpendek. Ata kalua lebih banyak bisa jadi itu yang terpanjang.
5. **Print** untuk menampilkan hasil yang terpendek mana dan yang Panjang.

SOAL 5

SOURCE CODE:

```
import re

from datetime import datetime


teks = """Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia
memiliki beberapa pahlawan nasional, seperti Pangeran Diponegoro (TL:
1785-11-11), Pattimura (TL: 1783-06-08) dan Ki Hajar Dewantara (1889-
05-02)."""


tanggal_list = re.findall(r'\d{4}-\d{2}-\d{2}', teks)

today = datetime.now()


for t in tanggal_list:

    t_date = datetime.strptime(t, "%Y-%m-%d")

    selisih = (today - t_date).days

    t_format = t_date.strftime("%d-%m-%Y")

    print(f"{t_format} selisih {selisih} hari")
```

INPUT:

```
import re
from datetime import datetime

teks = """Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan nasi

tanggal_list = re.findall(r'\d{4}-\d{2}-\d{2}', teks)

today = datetime.now()

for t in tanggal_list:
    t_date = datetime.strptime(t, "%Y-%m-%d")
    selisih = (today - t_date).days
    t_format = t_date.strftime("%d-%m-%Y")
    print(f"{t_format} selisih {selisih} hari")
```

OUTPUT:

```
17-08-1945 selisih 29465 hari
11-11-1785 selisih 87817 hari
08-06-1783 selisih 88704 hari
02-05-1889 selisih 50025 hari
PS C:\Users\Lenovo\alpro>
```

LANGKAH PENYELESAIAN:

1. Yang pertama kita menggunka fitur **datetime** untuk mengambil fitur tanggal dan waktu di python, khusus kelas datetime.
2. Yang kedua variabel **teks** untuk menyimpat data yang berisi YYYY-MM-DD.
3. Dan pada bagian tanggal_list \{4} adalah digit tahun \{2} untuk digit bulan \{2} untuk digit hari, dan findall untuk mengembalikansemuastringyangsesuaipola(matches).
4. Bagian **today** di dalamnya untuk sebagai perbandingan untuk menghitung selisi hari.
5. Pada bagian **t_date** mengubah teks menjadi objek tanggal %y=tahun, %m=bulan, %d=hari.
6. **Selisih** mengurangkan tanggal sekrang dan yang ada pada data dan .days untuk menghasilkan jumlah hari.
7. **t_format** untuk mengubah pola di dalam data menggunakan fungsi **strftime** lalu buat pola seperti yang kita inginkan.
8. **Print** untuk melihatkan hasil, bardasarkan tanggal sekrang.

SOAL 6

SOURCE CODE:

```
import re

import random

import string


teks = """Berikut adalah daftar email dan nama pengguna dari mailing
list:

vgraciatumimomor@mail.com dimiliki oleh Virjie Gracia Tumimomor

jheinc@gmail.co.id dimiliki oleh Jhein Lamasigi

vgloriatumimomor@getnada.com dimiliki oleh slamet slumut

padi@tokopedia.com dimiliki oleh toko tani

"""
```

```
emails = re.findall(r'[\w\.-]+@[\w\.-]+', teks)


for email in emails:

    username = email.split('@')[0]

    password = ''.join(random.choices(string.ascii_letters +
string.digits, k=8))


    print(f"{email} username: {username} , password: {password}")
```

INPUT:

```
import re
import random
import string

teks = """Berikut adalah daftar email dan nama pengguna dari mailing list:
vgraciatumimomor@mail.com dimiliki oleh Virjie Gracia Tumimomor
jheinc@gmail.co.id dimiliki oleh Jhein Lamasigi
vgloriatumimomor@getnada.com dimiliki oleh slamet slumut
padi@tokopedia.com dimiliki oleh toko tani
"""

emails = re.findall(r'[\w\.-]+@[\w\.-]+', teks)

for email in emails:
    username = email.split('@')[0]
    password = ''.join(random.choices(string.ascii_letters + string.digits, k=8))

    print(f"{email} username: {username} , password: {password}")
```

OUTPUT:

```
ers/Lenovo/alpro/soal6.py
vgraciatumimomor@mail.com username: vgraciatumimomor , password: RS1glBWz
jheinc@gmail.co.id username: jheinc , password: lNtTBG5d
vgloriatumimomor@getnada.com username: vgloriatumimomor , password: jaW6yrme
padi@tokopedia.com username: padi , password: AWOKxN5g
PS C:\Users\Lenovo\alpro>
```

PENJELASAN LANGKAH:

1. Pertama import **re** untuk mencari pola email, **random** untuk membuat password, dan **string** untuk mengambil huruf dan angka.
2. Dan variabel **teks** untuk menyimpan teks atau daftar” email.
3. Lanjut pada bagian emails **re.findall** digunakan unutk mencari dan semua teks yang cocok dengan pola dalam sebuah string.

4. **[\w\.-]** artrinya ambil karakter lebih dari satu.
5. **Username** untuk memisahkan email berdasarkan @ dan mengambil bagian depan sebagai username.
6. **Password** random.choices() digunakan unutk memilih beebrapa elem secara acak dari sekeumpulan data yang di dalamnya ada **ascii_letters** huruf besar +kecil, **digits** untuk angka k=8 itu adalah jumlah Panjang password.
7. **Print** untuk menampilkan hasil yang di dalamnya ada emal,username dan pw.